

Лабораторная работа

Тема «Объекты, связанные с тегами HTML. Визуальные эффекты»

Цель: рассмотреть примеры сценариев на языке JavaScript с использованием объектов, связанных с тегами HTML. Создание анимационных изображений.

Время выполнения: 2 часа.

Многие объекты языка JavaScript соответствуют тегам, формирующим HTML-документ. Каждый такой объект состоит во внутренней иерархии, поскольку все они имеют общий родительский объект браузер, который представлен объектом `window`.

Некоторые объекты языка JavaScript имеют потомков. В частности, гиперсвязь является объектом, наследованным из объекта `document`. В языке JavaScript наследованные объекты называются также свойствами. Например, множество гиперсвязей является свойством объекта `document`, а `links` – именем этого свойства. Таким образом, трудно провести четкую границу между объектами и свойствами. Гиперсвязь, являясь объектом, в то же время представляет собой свойство `links` объекта `document`.

Далее будут рассмотрены примеры, в которых будет создаваться объект `Image` и массив `images`.

1 Визуальные эффекты

1.1 Удаление, приближение и перемещение изображения в HTML-документе

С помощью сценариев JavaScript можно создавать простые визуальные эффекты: приближение и удаление изображения, перемещение изображения по странице и многое другое. Начнем с самых простых примеров, использующих только те средства, которые нам известны к настоящему времени.

Напишем сценарий, при выполнении которого создается эффект удаления изображения от зрителя, то есть изображение начинает уменьшаться в размерах.

Из предыдущих лабораторных работ известно, что объект **`document`** имеет свойство **`images`**, которое содержит ссылки на все изображения, расположенные в документе. Ссылки пронумерованы, начиная с нуля. Доступ к первому изображению на странице можно получить с помощью конструкции **`document.images[0]`**, ко второму – **`document.images[1]`** и так далее. Доступ к объекту **`Image`** можно получить с помощью значения `name` тега `<img>`.

Для реализации эффекта удаления будем использовать свойство **`height`** объекта **`Image`**. При каждом вызове функции **`succpict()`** изменяется размер выводимого изображения и тем самым достигается эффект удаления от зрителя. Функция **`setTimeout("succpict()", 500)`** производит повторный вызов функции **`succpict()`** через каждые полсекунды. Когда размер изображения изменится до заданного, движение прекратится. В начальный момент движение имеет вид, как на рисунке 1.

При наведении курсора мыши на рисунок изображение начинает удаляться от зрителя



Рисунок 1

HTML-код документа, содержащего сценарий решения задачи представлен ниже.

Листинг 18.1 – Сценарий удаляющегося изображения представлен ниже.

```
<!DOCTYPE html>
<html>
<head>
<title>Удаляющееся изображение</title>
<script>
var h = 200 //начальная высота
// эффект уменьшения изображения
var d= document
function succpict ()
{
  if (h > 100)
  {
    h=h-10
    d.images[0].height= h
    setTimeout("succpict()", 500)
  }
}
</script>
</head>
<body>
<H4>При наведении курсора мыши на рисунок<br>
изображение начинает удаляться от зрителя </H4>

</body></html>
```

Измените сценарий таким образом, чтобы при наведении курсора мыши на изображение оно начинало приближаться к зрителю.

Используя сценарии предыдущих примеров, создайте эффект пульсирующего изображения (имя файла `serde4ko.gif`): Изображение сначала увеличивается до определенного размера, затем уменьшается до заданного размера, потом снова увеличивается и так далее.

1.2 Чередование изображений с предварительной загрузкой

Напишем сценарий, при выполнении которого после загрузки страницы реализован визуальный эффект чередования изображений: на странице через равные промежутки времени появляются различные изображения. В ранее рассмотренных примерах сценарии писались без учета предварительной загрузки изображений.

Чтобы предварительно загрузить изображения, создадим массив **imgslide**: **imgslide=new Array()**. Каждый элемент массива представляет собой объект **Image**, который создается с помощью конструктора: **new Image()**. Первый элемент массива определяется так: **imgslide[0]=new Image()**. Свойство **src** только что созданного объекта задается в результате выполнения оператора присваивания: **imgslide[0].src= "155_p.jpg"**

Аналогично формируются и другие элементы массива. При работе сценария происходит обращение к элементам массива, и дополнительная загрузка изображений уже не потребуется.

Первоначальный вид HTML-документа представлен на рисунке 2.



Рисунок 2

HTML-код документа, содержащего сценарий решения задачи представлен ниже.

Листинг 18.2 – Сценарий смены изображений

```
<!DOCTYPE html>
<html>
<head>
<title>Смена изображений</title>
</head>
<script>
numimg=0
imgslide=new Array()
imgslide[0]=new Image()
imgslide[1]=new Image()
imgslide[2]=new Image()
imgslide[3]=new Image()
imgslide[4]=new Image()
imgslide[5]=new Image()
imgslide[6]=new Image()
imgslide[7]=new Image()
imgslide[8]=new Image()
imgslide[9]=new Image()
imgslide[10]=new Image()
imgslide[11]=new Image()
imgslide[12]=new Image()

imgslide[0].src="pojaspersidski.jpg"
imgslide[1].src="_1.jpg"
imgslide[2].src="_2.jpg"
imgslide[3].src="146_3.jpg"
imgslide[4].src="153_p.jpg"
```

```

imgslide[5].src="160_p.jpg"
imgslide[6].src="163_p.jpg"
imgslide[7].src="154_p.jpg"
imgslide[8].src="159_p.jpg"
imgslide[9].src="149_p.jpg"
imgslide[10].src="146_4.jpg"
imgslide[11].src="146_p_or_n.jpg"
imgslide[12].src="pojasvasilek.jpg"

function demoslides()
{ document.images[0].src=imgslide[numimg].src
numimg++
if(numimg==13)
numimg=0;
setTimeout("demoslides()", 1000)
}
</script>
<body bgcolor="#FFFFFF" >
<p></p>
<input type="button" value="Демонстрация слущких поясов"
onclick="demoslides()" />
</body></html>

```

Исправьте ошибку.

Фрагменты кода сценария, выделенные красным и фиолетовым цветом, запишите в более короткой форме.

1.3 Перемещение изображения в HTML-документе

В предыдущей задаче разные изображения появлялись на одном и том же месте на странице. Немного усложним задачу. Напишем сценарий реализации простой анимации: перемещение изображения слева направо.

Первоначальный вид HTML-документа представлен на рисунке 3.



Рисунок 3

HTML-код документа, содержащего сценарий решения задачи представлен ниже.

Листинг 18.3 – Сценарий перемещения изображения справа налево.

```

<!DOCTYPE html>
<html>
<head>
<title>Перемещение</TITLE>
<script type="text/javascript">
var stop=100
var n// номер кадра, в который вставляется изображение
var d= document
// предварительная загрузка изображений
imgempty = new Image()
imgempty.src = "7 l.jpg"
imganim = new Image()

```

```

imganim.src = "P.jpg"
// функция, осуществляющая анимацию
function anim1 ()
{var len= d.images.length
if (n != stop)
    {n = n+1
if (n >len)
    {n=1;
d.images[n-1].src=imganim.src
d.images[len-1].src=imgempty.src
    }
else
    {d.images[n-2].src=imgempty.src
d.images[n-1].src= imganim.src
    }
setTimeout("anim1()",500)
    }
}
// функция для задания начальной позиции
functionst ()
{ d.images[0].src=imganim.src
for (var i=1; i<= d.images.length-1; i++)
d.images[i].src= imgempty.src
}
</script>
</head>
<body>
<h4 align=left>Простая анимация</h4>
<hr>
<img src=7 1.jpg>
<img src=P.jpg>
<img src=P.jpg>
<img src=P.jpg>
<img src=P.jpg>
<img src=P.jpg>
<img src=P.jpg>
<img src=P.jpg>
<hr>
<FORM name="form1">
<input type="button" value=Начало onClick= "n = stop; st()">
<input type="button" value=Запустить onClick= "n=1; anim1()">
</form></body></html>

```

Для анимационного изображения будем использовать всего два рисунка. Первый рисунок – это изображение, которое перемещается. Второй рисунок должен соответствовать «пустому» изображению. Для этого можно создать одноцветный рисунок в формате jpg такого же размера, как и движущееся изображение. Для определенности будем считать, что имеется m кадров. Пусть n – номер кадра, в который помещается изображение. В следующий момент времени в кадр с номером n следует поместить «пустой» рисунок, в кадр с номером $n + 1$ изображение. Тем самым, как бы переместили изображение из кадра с номером $n + 1$ и создали эффект движения слева направо. Когда изображение появится в последнем кадре, его следует переместить в первый кадр. Выражение **document.images.length** определяет количество кадров. Сразу после загрузки документа изображение расположено в самом левом кадре. В остальных кадрах располагается «пустое» изображение.

При выполнении сценария перемещение изображения происходит небольшими скачками. Эти скачки можно сделать плавными, если увеличить число кадров на странице. Другой способ

– использование набора картинок, которые отражают при движении промежуточное состояние, соответствующее моменту, когда изображение переходит из одного кадра в другой. Чем больше кадров и чем точнее набор рисунков, тем плавней будет казаться движение по странице.

Следует помнить, что можно только менять уже имеющиеся на странице рисунки, но нельзя добавить новый рисунок или удалить существующий.

Разработайте сценарий создания графического баннера.

При разработке сценариев следует обратить внимание на вопросы корректности выполнения сценариев в разных браузерах.

2 Индивидуальные задания

Вариант 1. Разместите на странице четыре изображения, два поля ввода и кнопку *Переставить*. При нажатии на данную кнопку изображения с номерами, введенными в текстовые поля, должны поменяться местами.

Вариант 2. Организуйте последовательный просмотр изображений (не меньше четырех), используя кнопки *Предыдущее изображение* и *Следующее изображение*.

Вариант 3. Организуйте движение изображения справа налево.

Вариант 4. Организуйте движение изображения из левого верхнего угла окна к правому нижнему.

Вариант 5. Организуйте движение изображения из правого верхнего угла окна к левому нижнему.

Вариант 6. Организуйте движение изображения сверху вниз.

Вариант 7. Организуйте движение изображения снизу вверх.

Вариант 8. Разработать сценарий замены изображения по щелчку, рассчитанный на любое количество рисунков на странице.

Вариант 9. Подготовьте список участников олимпиады по информатике. При выборе фамилии должна появляться фотография участника. Фотография капитана команды должна быть с подписью «Капитан команды».

Вариант 10 Подготовьте три рисунка, на каждом из которых указано место (1, 2, 3). Напишите сценарий обработки результатов соревнований по теннису, при выполнении которого в документе появляется рисунок с номером присужденного места.

Вариант 11. Заданы три целых значения. Напишите сценарий, при выполнении которого определяется и выводится в документ информация о том, можно ли построить треугольник с длинами сторон, равными заданным значениям. Если треугольник построить можно, то требуется определить его вид: равносторонний, равнобедренный, разносторонний. При определении вида треугольника в документе должно появиться изображение треугольника соответствующего вида.

Вариант 12. Треугольник на плоскости задается координатами своих вершин. Напишите сценарий, в результате работы которого выясняется, лежит ли заданная точка на плоскости

внутри треугольника, вне его или на одной из сторон. В каждом из вариантов ответов должно быть выведено соответствующее графическое изображение.

Вариант 13. Два треугольника на плоскости задаются координатами своих вершин. Напишите сценарий, в результате работы которого определяется являются ли треугольники подобными. В каждом из вариантов ответов должно быть выведено соответствующее графическое изображение.

Вариант 14. Два круга на плоскости задаются координатами центра и радиуса. Напишите сценарий, в результате которого определяется положение кругов относительно друг друга. Варианты могут быть следующие:

- один круг лежит внутри другого;
- круги не имеют общих точек;
- круги пересекаются;
- круги имеют точку касания.

В каждом из вариантов ответов должно быть выведено соответствующее графическое изображение.

Вариант 15. Отрезок на плоскости задается координатами своих концов. Напишите сценарий, который для точки на плоскости, заданной собственными координатами, определяет:

Лежит ли заданная точка на отрезке;

Точка не лежит на отрезке, но находится на той же прямой, что и отрезок;

Точка не лежит на отрезке и не принадлежит прямой, содержащей отрезок.

Для каждого из вариантов ответа требуется вывести изображение, соответствующее ситуации.

ПОРЯДОК ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

1. Изучить теоретический материал [1, 321-323]. Вдумчиво изучить свойства объекта style, представленные в таблице [1, 321].
2. Понять и отладить скрипт, в сценарии которого используются свойства объекта style, [1, 322]. Проанализировать работу сценария.
3. Понять и отладить скрипты, рассмотренные в лабораторной работе.
4. Выполнить задания для самостоятельной работы, отмеченные фиолетовым цветом.
5. Ознакомится с методом drag and drop, реализацию которого демонстрирует HTML-документ drag_and_drop_flagi.html.
6. Составить обучающий сценарий, по которому на политической карте мира государственные флаги расставляются правильно.

Домашняя работа

7. Разработать сценарий индивидуального задания по вариантам.
 8. Ответить на контрольные вопросы [1, 323].
 9. Оформить отчет по лабораторной работе с указанием выполненных и невыполненных заданий. **Отлаженные скрипты подключить к сайту.**
 10. Защитить лабораторную работу.
-
1. Брылёва, А. А. Программные средства создания интернет-приложений : учеб. Пособие / А. А. Брылёва. – Минск : РИПО, 2022. – 483 с, : ил.

*Лабораторная работа составлена
преподавателем спецдисциплин
Еленой Иосифовной Шутько
сентябрь 2024 год*